

PROJECT REPORT

Basic To-Do List Application

with Categories and Deadlines

SUBMITTED TO

Rixi Lab Technologies

Bhatpar Rani, Deoria, Uttar Pradesh, INDIA 274506

Domain	Full Stack Development
Project Code	FS07P1A
Submission Date	August 2026

SUBMITTED BY

Intern ID: RL26080701

Bachelor of Computer Applications (7th Semester)
Brainware University

1. Project Objective

The primary objective of this project is to build a robust, backend-driven To-Do List application that supports complete task management. The system allows users to create, view, update, and delete tasks, while organizing them by category and tracking individual deadlines.

In strict adherence to the project specification (FS07P1A), this application performs all database operations without exposing a standard REST API. Instead, it relies on a Server-Side Rendering (SSR) approach using Node.js, Express, and MongoDB, ensuring that application logic and data integration are securely managed within a traditional backend-driven environment.

2. Technology Stack

The application was engineered using the following technologies:

Component	Technology Used	Purpose
Backend Environment	Node.js	Server runtime environment.
Web Framework	Express.js	Handling HTTP requests and URL routing.
Database	MongoDB	NoSQL database for persistent data storage.
ODM (Object Data Modeling)	Mongoose	Defining the task schema and querying the database.
View Engine	EJS	Rendering dynamic HTML pages on the server.
Frontend	HTML, CSS, JS	Structuring and styling the user interface.

3. System Architecture (MVC)

To ensure the codebase is scalable, maintainable, and highly organized, the application implements a strict Model-View-Controller (MVC) architectural pattern.

Model — `/models/Task.js`

Defines the data schema (title, category, deadline, status) and acts as the direct communication layer with the MongoDB database.

View — `/views/index.ejs`

The presentation layer. Data retrieved by the backend is securely injected into this HTML template on the server before being delivered as a fully rendered page to the user's browser.

Controller — `/controllers/taskController.js`

The operational core. It processes incoming HTTP requests, executes the necessary asynchronous database queries (CRUD operations), applies business logic, and passes the final output back to the View.

4. Implementation Details (CRUD Operations)

The core functionality revolves around Create, Read, Update, and Delete operations, executed directly via Express routes and form submissions:

- 1. Create** — A user submits an HTML form containing the task title, category, and deadline. The server captures this via a POST request and saves a new document to MongoDB.
- 2. Read** — Upon accessing the homepage (GET /), the server queries MongoDB for all existing tasks, sorts them by deadline, and renders them onto the UI.
- 3. Update** — Users can toggle a task's status between “Pending” and “Completed.” This triggers a POST request that identifies the task by its unique MongoDB `_id` and updates its status field.
- 4. Delete** — Users can permanently remove tasks. A POST request uses the `_id` to execute `findByIdAndDelete` against the database.

5. Custom Feature Enhancements

Beyond the baseline requirements, two advanced features were engineered to elevate the user experience and demonstrate more complex data manipulation:

- 1. Dynamic Category Filtering** — A dropdown filter on the frontend allows users to isolate tasks by category (Work, Study, Personal). The backend dynamically intercepts URL query parameters (e.g., `?category=Study`) and adjusts the MongoDB search parameters accordingly, rendering only the requested tasks.
- 2. Intelligent Deadline Alerts** — Algorithmic date-checking logic is embedded directly into the EJS rendering engine. The system performs real-time timestamp comparisons on every page load. Overdue tasks are flagged with bold red typography and a warning icon (⚠), while tasks due on the current day are highlighted in orange (📅).

6. Cloud Infrastructure & Deployment

To demonstrate a comprehensive understanding of full-stack development and DevOps principles, the completed application was provisioned and deployed to a live cloud environment.

- **Cloud Provider:** Amazon Web Services (AWS)
- **Compute Resource:** Amazon EC2 (Ubuntu 24.04 LTS instance)
- **Environment Configuration:** The Node.js runtime and MongoDB server were installed and configured natively on the virtual machine.
- **Process Management:** PM2 was used to daemonize the application, ensuring maximum uptime and automated recovery in the event of server reboots.

- **Networking:** AWS Security Groups were configured to expose standard web traffic ports while restricting unauthorized access.

7. Conclusion

The development of the “Basic To-Do List Application with Categories and Deadlines” successfully fulfills the requirements outlined for project FS07P1A by Rixi Lab Technologies. By applying a disciplined MVC architecture, integrating Server-Side Rendering without a standard API layer, adding user-centric features, and executing a full AWS cloud deployment, this project demonstrates a strong, practical grasp of modern backend engineering and database integration.